6.1 Чтение и запись файлов

Содержание

1. Чтение файла
2. Запись в файл
3. Чтение большого файла
4. Запись с использованием буферизации
5. Чтение CSV файла
6. Запись в CSV файл
7. Чтение JSON файла
8. Запись в JSON файл

1. Чтение файла

Чтение файла в Go выполняется с использованием пакета `os` . Пример ниже читает всё содержимое файла в память.

Файл:

```
file.txt
12345aaa
```

Код: main.go

```
package main

import (
 "fmt"
 "os"
 "time"
)

func main() {
   data, err := os.ReadFile("file.txt")
   if err != nil {
      fmt.Println("Ошибка чтения файла:", err)
      return
   }
   fmt.Println(string(data))


   time.Sleep(time.Minute)


}
```

Объяснение:

- `os.ReadFile` читает весь файл в срез байтов.
- Обработка ошибок необходима для случаев, таких как отсутствие файла или проблемы с правами доступа.
- Содержимое преобразуется в строку для отображения.

2. Запись в файл

Запись в файл включает создание или открытие файла и запись данных с использованием пакета `os`.

Файл: `output.txt`

```
Привет, Go!
```

Код: main.go

```go
package main

import (
  "fmt"
  "os"
)

func main() {
   file, err := os.Create("output.txt")
   if err != nil {
      fmt.Println("Ошибка создания файла:", err)
      return
   }
   defer file.Close()

   _, err = file.WriteString("Привет, Go!")
   if err != nil {
      fmt.Println("Ошибка записи в файл:", err)
      return
   }
}
```

Объяснение:

- `os.Create` создаёт или обрезает файл.
- `file.WriteString` записывает строку в файл.
- `defer file.Close()` гарантирует закрытие файла после записи.
- Обработка ошибок критически важна для надёжной работы с файлами.

3. Чтение большого файла

Для больших файлов чтение по частям или построчно более эффективно с точки зрения памяти, чем загрузка всего файла в память.

Файл: `largefile.txt`

```
Давайте спроектируем с нуля планировщик Go — начнём с самой простой и понятной
наивной реализации, а затем шаг за шагом будем разбираться, какие изъяны в ней
есть, и придумывать как их решать, постепенно усложняя общую модель.
Это один из лучших способов понять сложную систему или концепцию — пройти
путь её поэтапного проектирования. Система сложна, осознать её очень
непросто, но мы разобьём её на простые шаги, понять которые очень легко.
После этого пазл сам собой сложится в голове, и общая картина системы будет
для вас такой же простой и очевидной.
```

Код: main.go

```
package main

import (
"fmt"
"io"
"os"
)

func main() {
   file, err := os.Open("largefile.txt")
   if err != nil {
      fmt.Println("Ошибка открытия файла:", err)
      return
   }
   defer file.Close()

   buffer := make([]byte, 256) for {
      n, err := file.Read(buffer)
      if err != nil {
         if err != io.EOF {
            fmt.Println("Ошибка чтения файла:", err)
         }
         break
      }

      fmt.Println("----------------------------------------")
      fmt.Printf("Прочитано %d байт: %s\n", n, string(buffer[:n]))
   }
}
```

Объяснение:

- `os.Open` открывает файл для чтения.
- Буфер размером 256 байт используется для чтения файла по частям.
- Цикл продолжается до достижения `io.EOF` (конец файла).

Этот подход эффективен по памяти для больших файлов.

Альтернативный подход (закомментирован в коде):

Использование `bufio.NewScanner` для чтения построчно, подходит для текстовых файлов с чёткими разделителями строк.

4. Запись с использованием буферизации

Буферизованная запись повышает производительность, уменьшая количество прямых операций с диском.

Файл: `output.txt`

```
Пример буферизованной записи
```

Код: main.go

```
package main

import (
   "bufio"
   "fmt"
   "os"
)

func main() {
   file, err := os.Create("output.txt")
   if err != nil {
      fmt.Println("Ошибка создания файла:", err)
      return
   }
   defer file.Close()

   writer := bufio.NewWriter(file)
   defer writer.Flush()

   _, err = writer.WriteString("Пример буферизованной
   записи")
   if err != nil {
      fmt.Println("Ошибка записи в файл:", err)
   }
}
```

Объяснение:

- `bufio.NewWriter` создаёт буферизированный писатель.
- `writer.Flush()` гарантирует, что все буферизированные данные записаны в файл.

Буферизованная запись эффективна для частых или больших операций записи.

5. Чтение CSV файла

Чтение CSV файлов осуществляется с использованием пакета `encoding/csv` для разбора структурированных данных.

Файл: `data.csv`

```
id,username
,salary
1,oleg,2456
4
2,alex,4675
4
```

Код: main.go

```
package main

import (
   "encoding/csv"
   "fmt"
   "io"
   "os"
)

func main() {
   file, err := os.Open("data.csv")
   if err != nil {
      fmt.Println("Ошибка открытия файла:", err)
      return
   }
   defer file.Close()
   reader := csv.NewReader(file)
   for {
      record, err := reader.Read()
      if err != nil {
         if err == io.EOF {
          break
         }
         fmt.Println("Ошибка чтения CSV:", err)
         return
      }
      fmt.Println("Запись:", record)
   }
}
```

Объяснение:

- `csv.NewReader` создаёт читатель CSV из файла.
- `reader.Read()` читает одну запись (строку) за раз.
- Цикл прекращается при достижении `io.EOF`.

Альтернативный подход (закомментирован) использует `reader.ReadAll()` для чтения всех записей сразу.

6. Запись в CSV файл

Запись в CSV файл осуществляется с использованием пакета `encoding/csv` для создания структурированных данных.

Файл: `output.csv`

```
Имя,Фамилия,Возраст
Иван,Иванов,30
Мария,Петрова,25
```

Код: main.go

```
package main

import (
   "encoding/csv"
   "fmt"
   "os"
)

func main() {
   file, err := os.Create("output.csv")
   if err != nil {
      fmt.Println("Ошибка создания файла:", err)
      return
   }
   defer file.Close()

   writer := csv.NewWriter(file)
   defer writer.Flush()

   records := [][]string{
      {"Имя", "Фамилия", "Возраст"},
      {"Иван", "Иванов", "30"},
      {"Мария", "Петрова", "25"},
   }

   if err = writer.WriteAll(records); err != nil {
      fmt.Println("Ошибка записи:", err)
      return
   }
```

Объяснение:

- `csv.NewWriter` создаёт писатель CSV.
- `writer.WriteAll` записывает несколько записей сразу.
- `writer.Flush()` гарантирует, что все данные записаны в файл.

7. Чтение JSON файла

Чтение JSON файлов включает декодирование данных JSON в структуры Go с использованием пакета `encoding/json` .

Файл: `cfg.json`

```
{
 "name": "Иван
 Иванов",
 "age": 30,
 "email":
 "ivan@example.com"
 , "cars": ["audi",
 "bmw"]
}
```

Код: main.go

```
package main

import (
   "encoding/json" "fmt"
   "os"
)

type Person struct {
Name string   `json:"name"`
Age int       .`json:"age"`
Email string  `json:"email,omitempty"
Cars []string `json:"cars,omitempty"`
}

func main() {
   data, err := os.ReadFile("cfg.json")
   if err != nil {
      fmt.Println("Ошибка чтения файла:", err)
      return
   }

   var person Person
   err = json.Unmarshal(data, &person)
   if err != nil {
      fmt.Println("Ошибка декодирования JSON:", err)
      return
   }

   fmt.Printf("Имя: %s\nВозраст: %d\nEmail: %s\n", person.Name, person.Age, person.Email)
}
```

Объяснение:

- `os.ReadFile` читает JSON файл.
- `json.Unmarshal` декодирует данные JSON в структуру `Person`.
- Теги структуры (`json:"name"`) сопоставляют ключи JSON с полями структуры.
- Тег `omitempty` пропускает пустые поля при кодировании/декодировании.

8. Запись в JSON файл

Запись JSON файлов включает кодирование структур Go в формат JSON.

```
package main

import (
    "encoding/json"
    "fmt"
    "os"
)

func main() {
    type Person struct {
        Name string `json:"name"`
        LastName string `json:"lastName,omitempty"`
        Age  int  `json:"age"`
    }

    people := []Person{
        {Name: "Иван", Age: 30, LastName: "Иванов"},
        {Name: "Мария", Age: 25},
    }

    file, err := os.Create("people.json")
    if err != nil {
        fmt.Println("Ошибка создания файла:", err)
        return
    }
    defer file.Close()

    encoder := json.NewEncoder(file)
    if err := encoder.Encode(people); err != nil {
    fmt.Println("Ошибка записи JSON:", err)

    }
```

Объяснение:

- Создаётся срез структур `Person`.
- `json.NewEncoder` записывает JSON напрямую в файл.
- `encoder.Encode` сериализует данные в формат JSON.
- `omitempty` гарантирует, что необязательные поля пропускаются, если они пусты.

Итог

Были рассмотрены основные техники работы с файлами в Go:

- **Чтение**: Используйте `os.ReadFile` для маленьких файлов, `bufio.Scanner` или чтение по частям для больших файлов.
- **Запись**: Используйте `os.Create` и `WriteString` для простых операций записи, или `bufio.NewWriter` для буферизированной записи.
- **Обработка CSV**: Используйте `encoding/csv` для чтения и записи структурированных данных CSV.
- **Обработка JSON**: Используйте `encoding/json` для кодирования и декодирования структурированных данных.
- **Лучшие практики**: Всегда обрабатывайте ошибки, используйте `defer` для очистки ресурсов и выбирайте подходящие методы для размера файла и требований производительности.

Для дополнительной информации обратитесь к документации Go по пакетам `os`, `bufio`, `encoding/csv` и `encoding/json`.